

Conceitos preliminares

Cotas Inferiores

Estudo do Problema de ordenação

Algoritmos:

- Seleção
- Quicksort
- Mergesort
- Inserção

Conclusão: o melhor dentre os algoritmos vistos (**Mergesort**) foi capaz de ordenar n números em tempo $O(n \log n)$.

Dúvida: existe algum algoritmo para o problema de ordenação que tenha desempenho superior ao **Mergesort**? Mais precisamente queremos saber qual a menor complexidade possível entre todos os algoritmos que resolvem o problema de ordenação.

Consequência: caso seja possível obter uma resposta a essa pergunta, estaremos em condições de falar a respeito da **complexidade do problema de ordenação**.

Cotas Inferiores

Obter a complexidade de um problema (isto é, obter um algoritmo para ele cuja complexidade seja uma **cota inferior** entre as complexidades de todos os algoritmos possíveis para o problema) é, em geral, uma tarefa não trivial.

Definição: A complexidade de um **problema P** tem **cota inferior** dada por $\Omega(f(n))$ se existe uma constante k tal que todos os algoritmos que resolvem **P** requerem pelo menos $kf(n)$ passos para resolver alguma instância de tamanho n .

A complexidade do problema **P** é $\theta(f(n))$ se, além disso, existir um algoritmo de complexidade $O(f(n))$ para **P** (neste caso, dizemos que esse **algoritmo** é ótimo para **P**).

Cotas Inferiores

Resumo:

$\Omega(f(n))$:

- cota inferior de um problema

$O(f(n))$:

- complexidade de um algoritmo que resolve o problema

$\theta(f(n))$:

- complexidade do problema, caso exista pelo menos um algoritmo que atinja esse limite. Esse algoritmo é ótimo

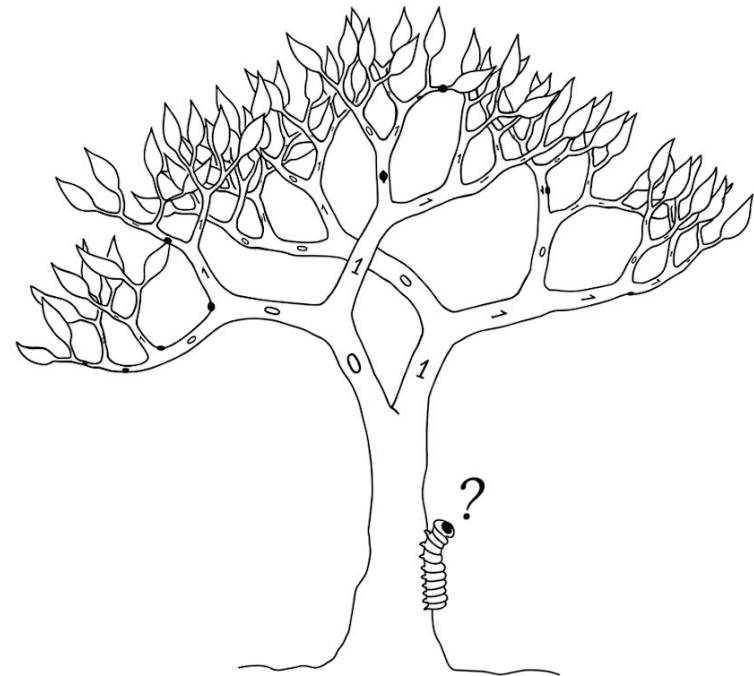
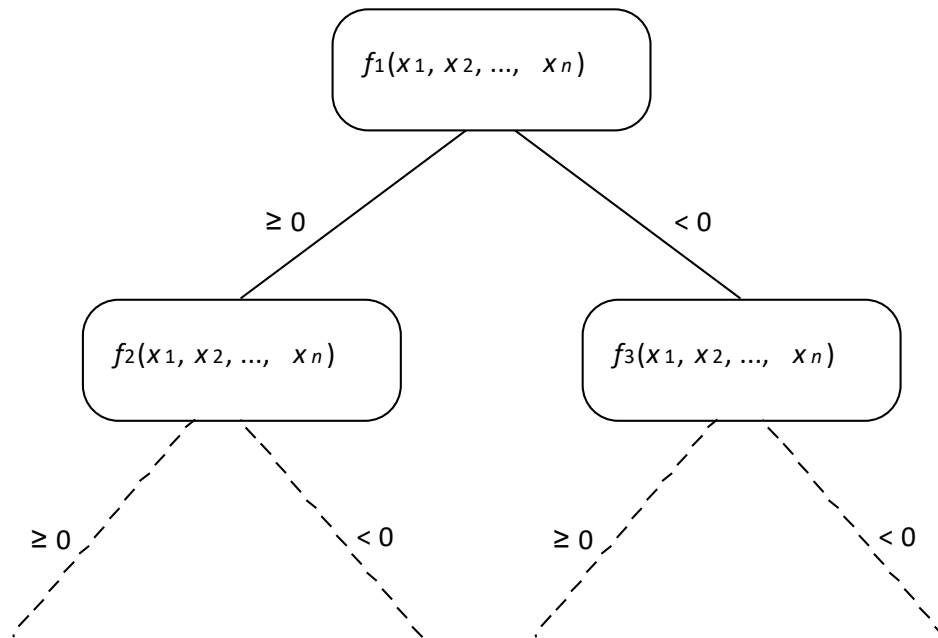
Cotas Inferiores

- A obtenção de resultados neste sentido está relacionada ao **modelo de complexidade computacional** adotado.
- Uma escolha adequada para Geometria Computacional é a classe das **árvores de decisões algébricas** (todos os algoritmos de ordenação vistos podem ser simulados usando tal modelo, já que eles utilizam apenas comparações).
- Para os **problemas** que podem ser inseridos em um modelo de **árvores de decisões algébricas**, é possível obter resultados que estabelecem cotas inferiores. O **problema de ordenação** está inserido neste contexto.

Teorema: Sob o modelo de árvores algébricas de decisão, o problema de ordenação tem complexidade $\theta(n \log n)$. Assim, o algoritmo **Mergesort**, que tem essa complexidade, é **ótimo** para o problema.

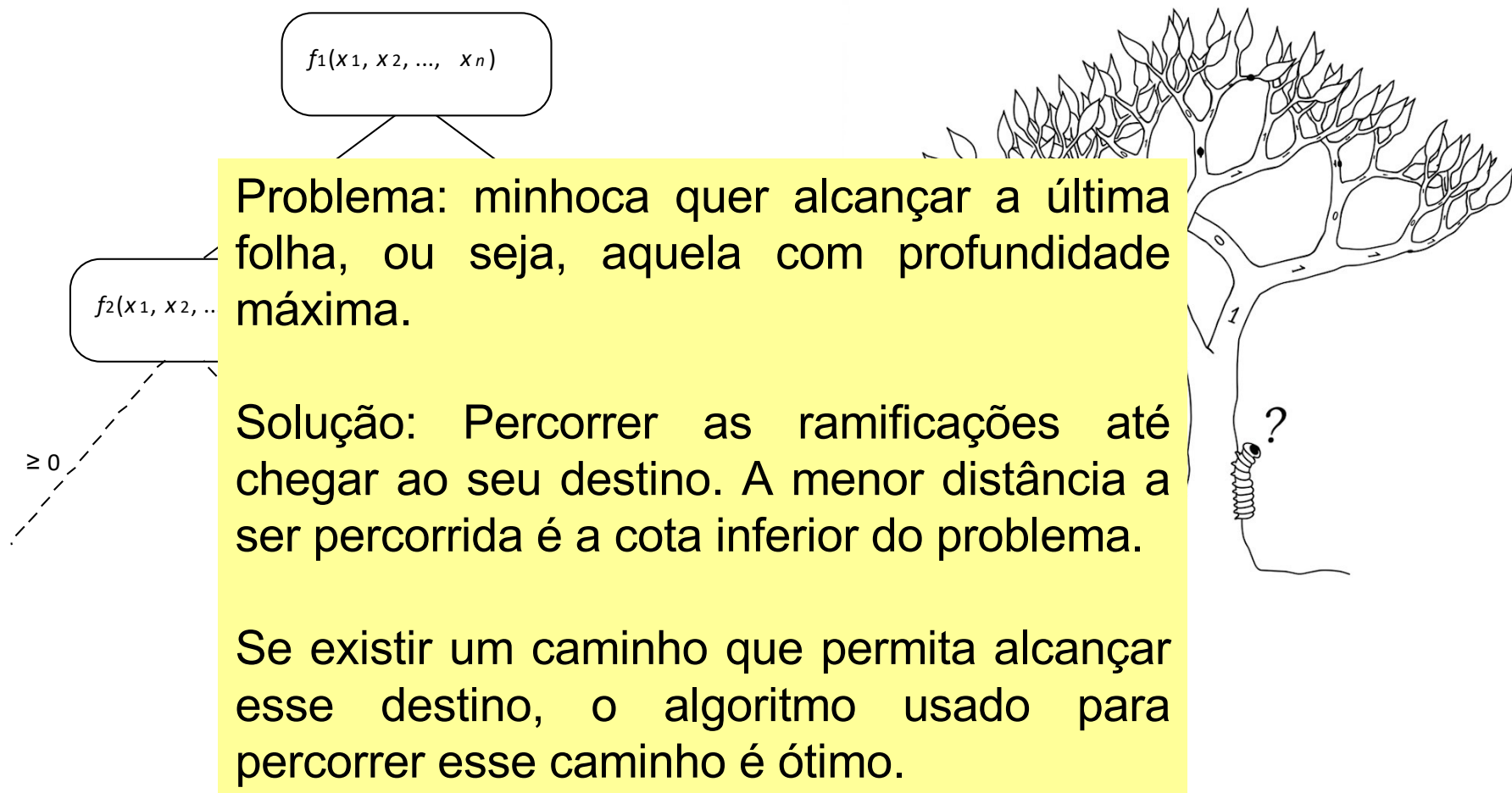
Cotas Inferiores

Prova: A complexidade de uma árvore de decisão está relacionada à sua profundidade p , isto é, a profundidade máxima de uma de suas folhas (elementos que não possuem ramos).



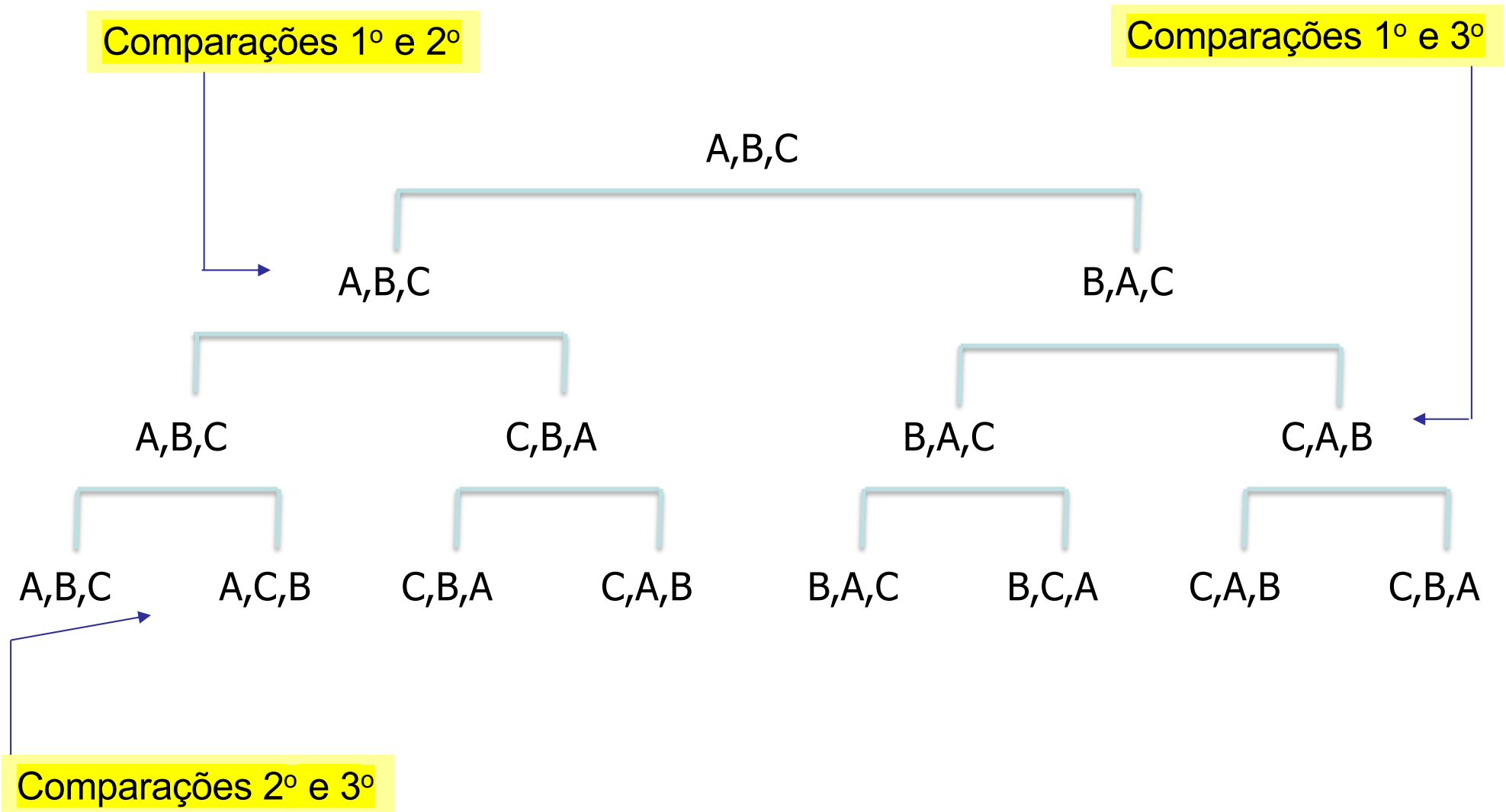
Cotas Inferiores

Prova: A complexidade de uma árvore de decisão está relacionada à sua profundidade p , isto é, a profundidade máxima de uma de suas folhas (elementos que não possuem ramos).



Cotas Inferiores

- Problema de ordenação: cada uma das folhas da árvore de decisão deverá estar associada a uma possível ordenação dos n elementos.



Cotas Inferiores

- Problema de ordenação: cada uma das folhas da árvore de decisão deverá estar associada a uma possível ordenação dos n elementos.
- Como existem $n!$ ordenações possíveis para um conjunto com n elementos, a árvore possui no mínimo $n!$ folhas.
- Mas o número máximo possível de folhas em uma árvore de decisões de profundidade p é obtido quando todas as folhas ocorrem a esta profundidade e cada nó que não é uma folha tem exatamente 2 descendentes. Neste caso, o número de folhas é igual ao número de nós à profundidade p , que é 2^p .
- Logo, a profundidade p de uma árvore de decisão que possui pelo menos $n!$ folhas satisfaz:

$$2^p \geq n! \rightarrow p \geq \log_2(n!)$$

Cotas Inferiores

- A aproximação de Stirling para $n!$ fornece

$$n! \approx \sqrt{2\pi n} (n/e)^n$$

ou seja:

$$\log_2(n!) \approx \log_2(\sqrt{2\pi n}) + n \log_2 n - n \log_2 e \geq kn \log_2 n$$

- Logo, a complexidade p tem cota inferior $\Omega(n \log n)$.
- Como existe um algoritmo de complexidade $O(n \log n)$, concluímos que a complexidade do problema de ordenação é $\theta(n \log n)$.

Redução

Os resultados obtidos a respeito do problema de ordenação são úteis para estabelecer **cotas inferiores** para problemas de geometria computacional, particularmente para os problemas de determinação do fecho convexo de um conjunto de pontos e obtenção de uma triangulação de um conjunto de pontos do plano.

Para tal, será utilizado um mecanismo que consiste em **reduzir um problema a outro**.

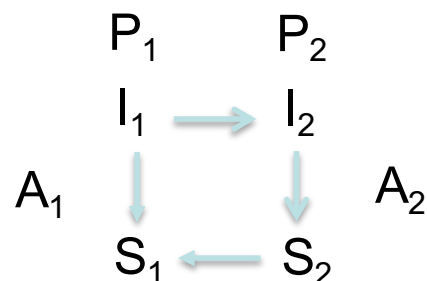
Redução

Os resultados obtidos a respeito do problema de ordenação são úteis para estabelecer **cotas inferiores** para problemas de geometria computacional, particularmente para os problemas de determinação do fecho convexo de um conjunto de pontos e obtenção de uma triangulação de um conjunto de pontos do plano.

Para tal, será utilizado um mecanismo que consiste em **reduzir um problema a outro**.

Sejam P_1 e P_2 dois problemas. Suponha que sabemos transformar instâncias de P_1 em instâncias de P_2 e soluções de P_2 em soluções de P_1 .

Nesse caso, dizemos que podemos reduzir P_1 a P_2 , isto é, para resolver uma instância de P_1 basta resolver uma instância adequada de P_2 . Essa situação pode ser resumida no seguinte diagrama:

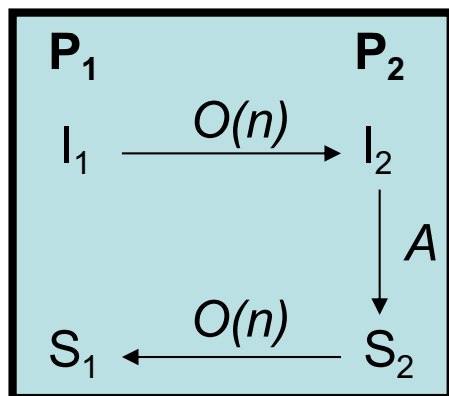


Se existe um algoritmo $A_2: I_2 \rightarrow S_2$ que resolve P_2 , então existe um algoritmo $A_1: I_1 \rightarrow I_2 \rightarrow S_2 \rightarrow S_1$ que resolve P_1 .

Redução

Definição: Um problema P_1 é **reduzível** (em tempo linear) a um problema P_2 se dada uma instância I_1 de tamanho n de P_1 for possível, em tempo $O(n)$, obter uma instância I_2 de P_2 tal que a resposta de P_1 para I_1 possa ser obtida, em tempo $O(n)$, a partir da resposta de P_2 para I_2 . Neste caso, escrevemos:

$$P_1 \propto_{O(n)} P_2 \quad \text{ou} \quad P_1 \propto P_2$$



A ideia intuitiva desta definição é a seguinte: se P_1 é reduzível a P_2 , então P_1 é pelo menos tão fácil de se resolver quanto P_2 .

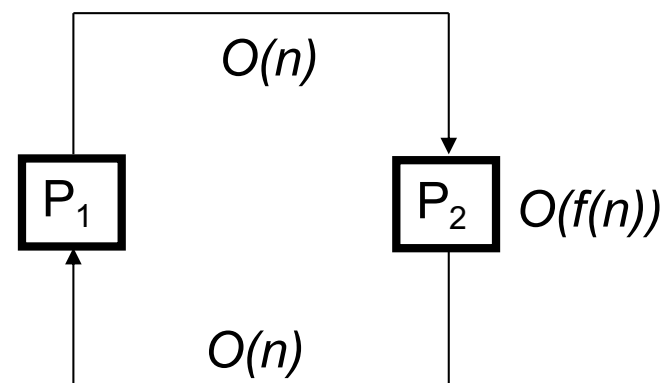
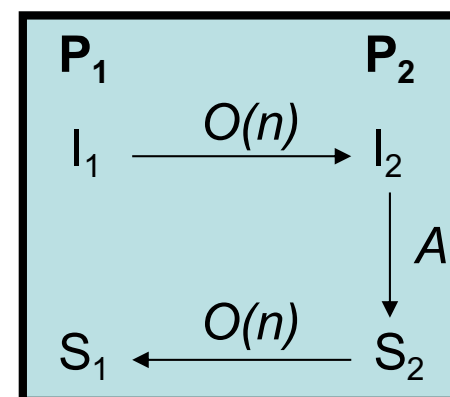
Redução

Teorema: Suponhamos que P_1 e P_2 sejam problemas tais que $P_1 \propto P_2$. Então:

- a) Se existe um algoritmo que resolve P_2 em $O(f(n))$ passos, então também existe um algoritmo que resolve P_1 em $O(f(n))$ passos.

Prova: Um algoritmo $O(f(n))$ para P_1 consiste simplesmente em concatenar:

- o algoritmo que converte instâncias de P_1 em instâncias de P_2 ($O(n)$);
- o algoritmo $O(f(n))$ para P_2 ;
- o algoritmo que converte a resposta de P_2 na resposta correspondente de P_1 ($O(n)$).



Redução

- b) Se qualquer algoritmo para P_1 requer, no pior caso, $\Omega(f(n))$ passos, então qualquer algoritmo para P_2 também requer, no pior caso, $\Omega(f(n))$ passos.

Prova: Se houvesse um algoritmo capaz de resolver P_2 em tempo inferior a $f(n)$, pela parte (a) também haveria um tal algoritmo para P_1 , o que contradiz a hipótese de P_1 requerer $\Omega(f(n))$ passos.

Resumo:

- Podemos transferir para P_1 quaisquer limites superiores que sejam conhecidos para P_2 .
- O poder da redução está principalmente na recíproca: podemos transferir para P_2 quaisquer limites inferiores que sejam conhecidos para P_1 .

Redução

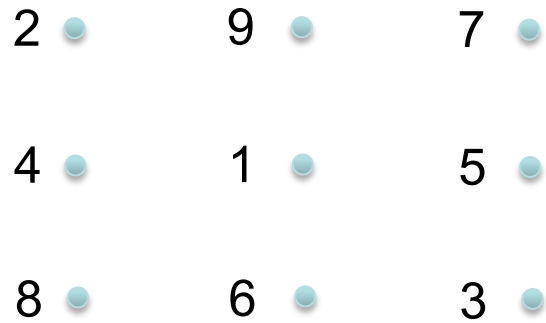
Exemplo: Relação ordenação/triangulação

Redução

Exemplo: Relação ordenação/triangulação

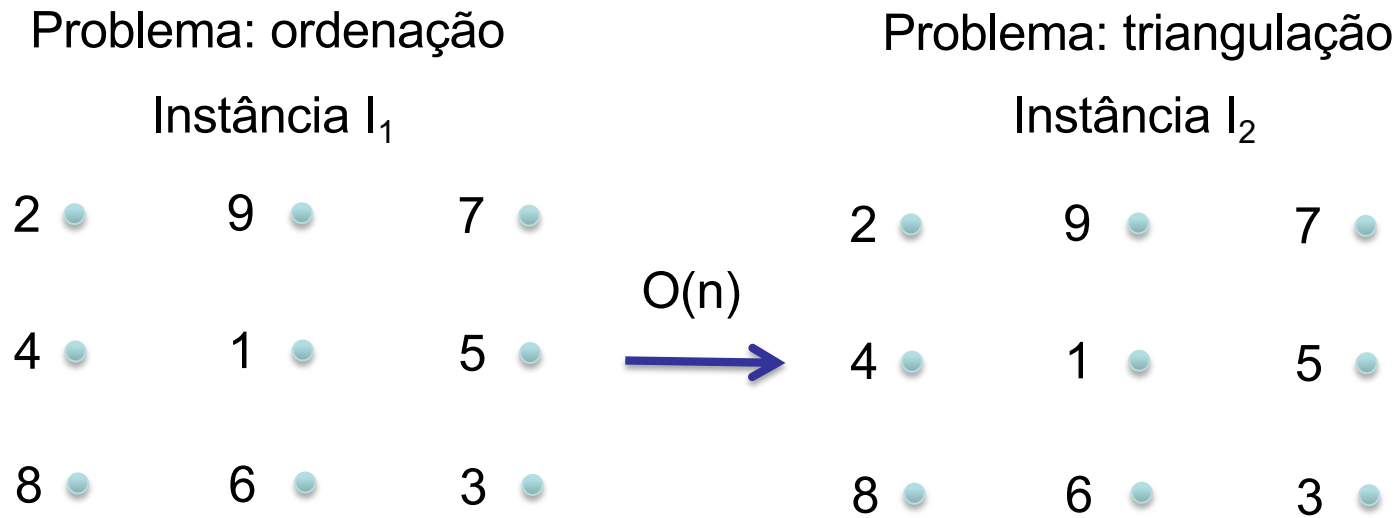
Problema: ordenação

Instância I_1



Redução

Exemplo: Relação ordenação/triangulação



Redução

Exemplo: Relação ordenação/triangulação

Problema: ordenação

Instância I_1

2	9	7
4	1	5
8	6	3

$O(n)$



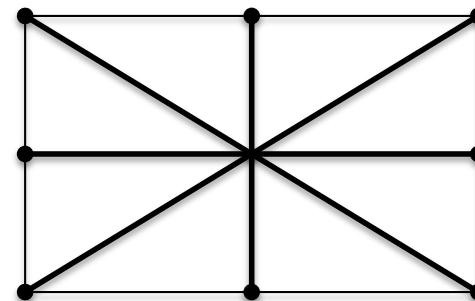
Problema: triangulação

Instância I_2

2	9	7
4	1	5
8	6	3

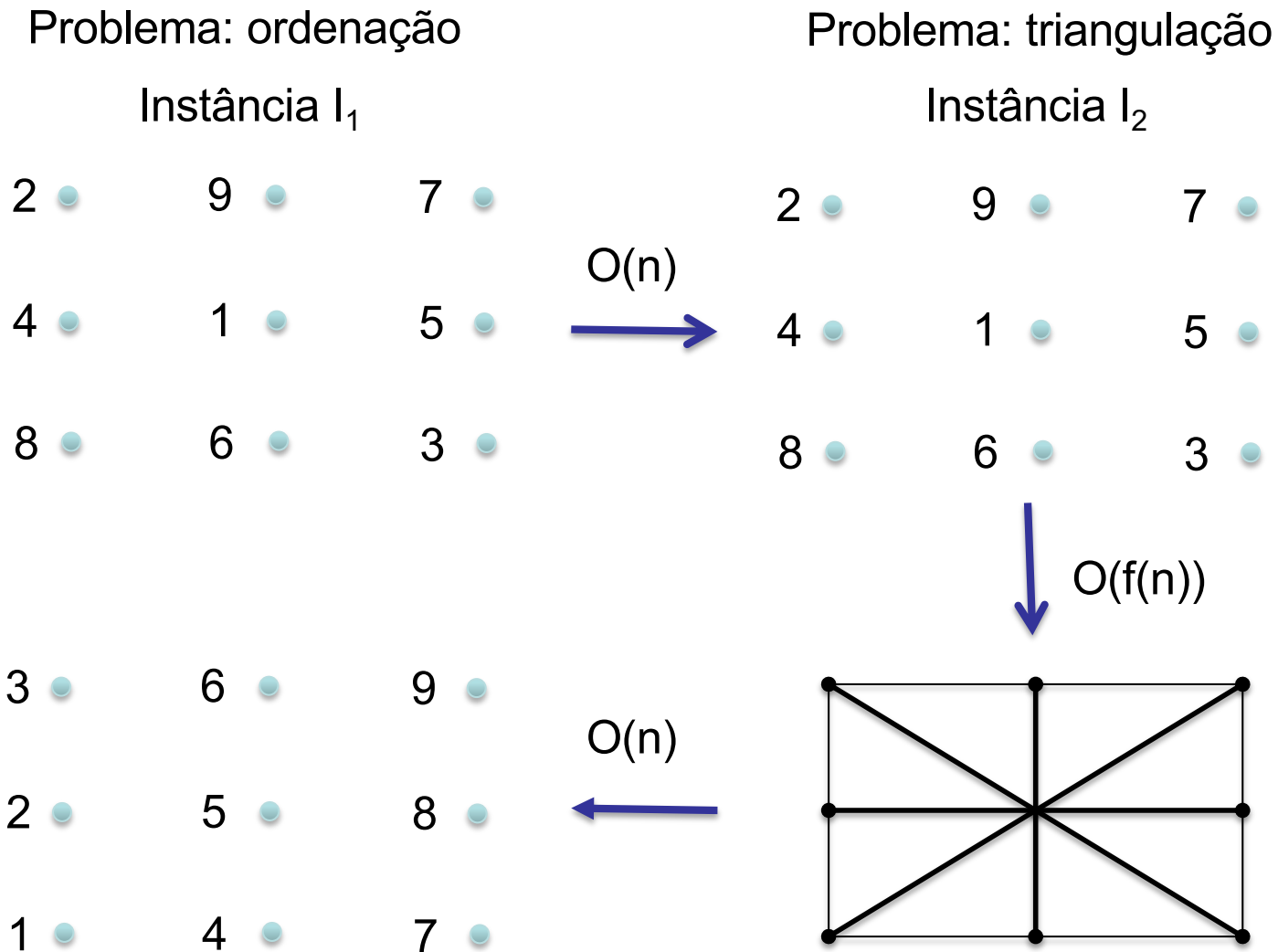


$O(f(n))$



Redução

Exemplo: Relação ordenação/triangulação



Redução

Exemplo: Relação ordenação/triangulação (conclusões)

$$\text{ordenação} \propto \text{triangulação}$$

- 1) Se existe um algoritmo que resolve **triangulação** em $O(f(n))$ passos, então também existe um algoritmo que resolve **ordenação** em $O(f(n))$ passos.
- 2) Se qualquer algoritmo para **ordenação** requer, no pior caso, $\Omega(n \log n)$ passos, então qualquer algoritmo para **triangulação** também requer, no pior caso, $\Omega(n \log n)$ passos.

Cota inferior do problema de triangulação: ???

Redução

Exemplo: Relação ordenação/triangulação (conclusões)

$$\text{ordenação} \propto \text{triangulação}$$

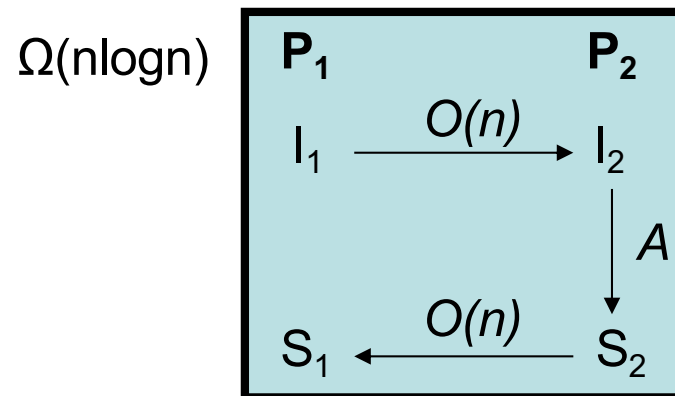
- 1) Se existe um algoritmo que resolve **triangulação** em $O(f(n))$ passos, então também existe um algoritmo que resolve **ordenação** em $O(f(n))$ passos.
- 2) Se qualquer algoritmo para **ordenação** requer, no pior caso, $\Omega(n \log n)$ passos, então qualquer algoritmo para **triangulação** também requer, no pior caso, $\Omega(n \log n)$ passos.

Cota inferior do problema de triangulação: $\Omega(n \log n)$

Exemplos

1) Sejam P_1 e P_2 dois problemas tais que $P_1 \propto P_2$ e suponha que P_1 tem cota inferior $\Omega(n \log n)$. Quais das seguintes afirmações são verdadeiras?

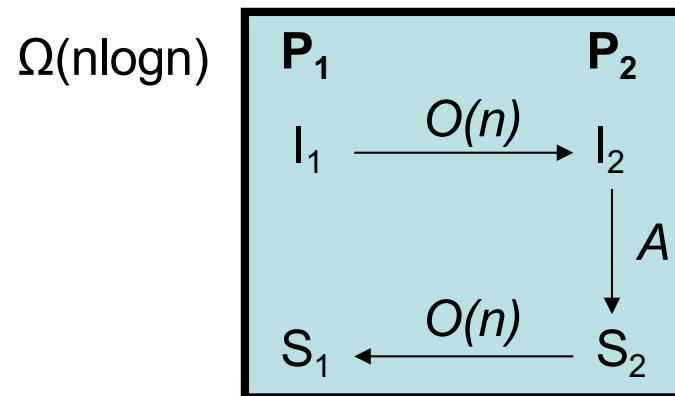
- a) $\Omega(n \log n)$ é também uma cota inferior para P_2 ;
- b) Todo algoritmo que resolve P_1 pode ser usado para resolver P_2 ;
- c) Todo algoritmo que resolve P_2 pode ser usado para resolver P_1 ;
- d) O problema P_2 tem complexidade $\theta(n \log n)$.



Exemplos

1) Sejam P_1 e P_2 dois problemas tais que $P_1 \propto P_2$ e suponha que P_1 tem cota inferior $\Omega(n \log n)$. Quais das seguintes afirmações são verdadeiras?

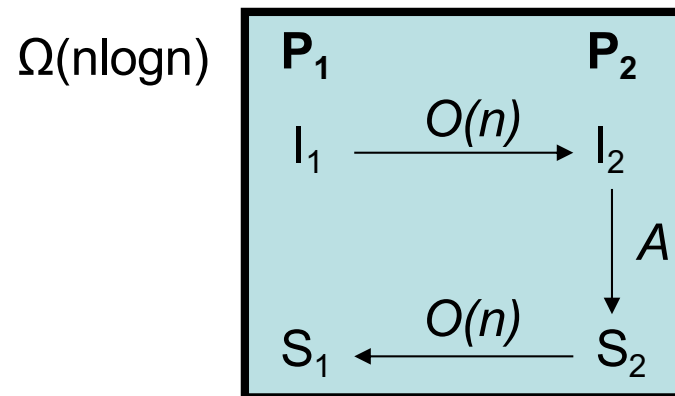
- a) $\Omega(n \log n)$ é também uma cota inferior para P_2 ; (V)
- b) Todo algoritmo que resolve P_1 pode ser usado para resolver P_2 ;
- c) Todo algoritmo que resolve P_2 pode ser usado para resolver P_1 ;
- d) O problema P_2 tem complexidade $\theta(n \log n)$.



Exemplos

1) Sejam P_1 e P_2 dois problemas tais que $P_1 \propto P_2$ e suponha que P_1 tem cota inferior $\Omega(n \log n)$. Quais das seguintes afirmações são verdadeiras?

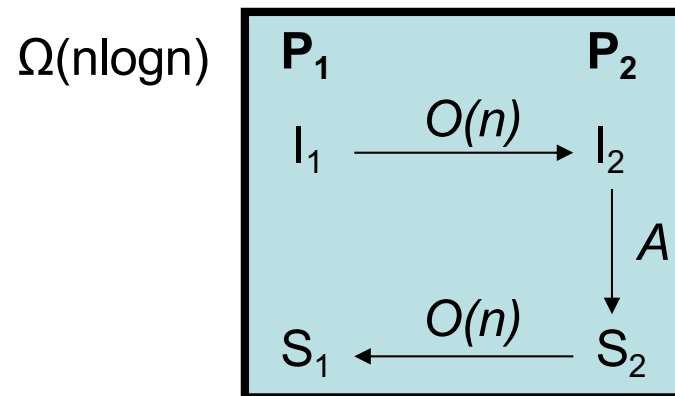
- a) $\Omega(n \log n)$ é também uma cota inferior para P_2 ; (V)
- b) Todo algoritmo que resolve P_1 pode ser usado para resolver P_2 ; (F)
- c) Todo algoritmo que resolve P_2 pode ser usado para resolver P_1 ;
- d) O problema P_2 tem complexidade $\theta(n \log n)$.



Exemplos

1) Sejam P_1 e P_2 dois problemas tais que $P_1 \propto P_2$ e suponha que P_1 tem cota inferior $\Omega(n \log n)$. Quais das seguintes afirmações são verdadeiras?

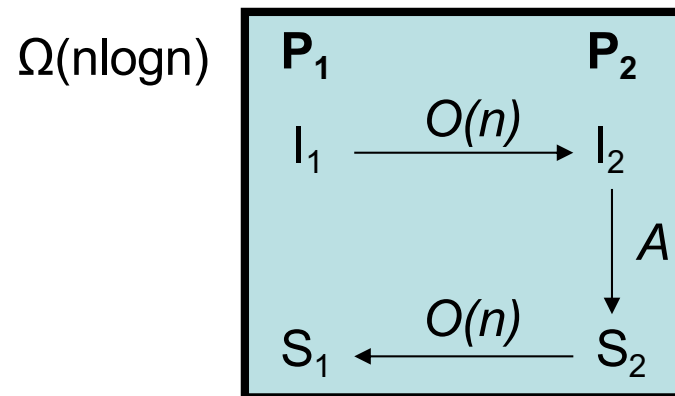
- a) $\Omega(n \log n)$ é também uma cota inferior para P_2 ; (V)
- b) Todo algoritmo que resolve P_1 pode ser usado para resolver P_2 ; (F)
- c) Todo algoritmo que resolve P_2 pode ser usado para resolver P_1 ; (V)
- d) O problema P_2 tem complexidade $\theta(n \log n)$.



Exemplos

1) Sejam P_1 e P_2 dois problemas tais que $P_1 \propto P_2$ e suponha que P_1 tem cota inferior $\Omega(n \log n)$. Quais das seguintes afirmações são verdadeiras?

- a) $\Omega(n \log n)$ é também uma cota inferior para P_2 ; (V)
- b) Todo algoritmo que resolve P_1 pode ser usado para resolver P_2 ; (F)
- c) Todo algoritmo que resolve P_2 pode ser usado para resolver P_1 ; (V)
- d) O problema P_2 tem complexidade $\theta(n \log n)$. (F)

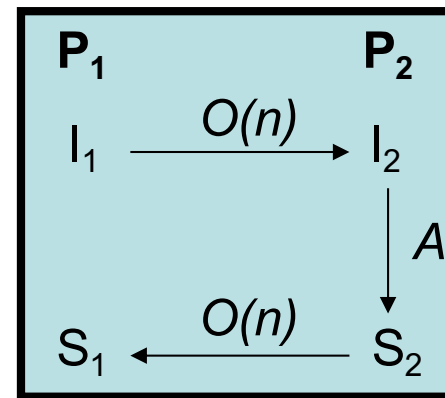


Exemplos

1) Sejam P_1 e P_2 dois problemas tais que $P_1 \propto P_2$ e suponha que P_1 tem cota inferior $\Omega(n \log n)$. Quais das seguintes afirmações são verdadeiras?

- a) $\Omega(n \log n)$ é também uma cota inferior para P_2 ; (V)
- b) Todo algoritmo que resolve P_1 pode ser usado para resolver P_2 ; (F)
- c) Todo algoritmo que resolve P_2 pode ser usado para resolver P_1 ; (V)
- d) O problema P_2 tem complexidade $\theta(n \log n)$. (F)

2) Sejam P_1 e P_2 dois problemas tais que um deles tem cota inferior $\Omega(n^k)$, para algum $k > 1$, e o outro é solucionável em tempo $O(n \log n)$. Se P_1 é redutível a P_2 em tempo linear, quem é quem?

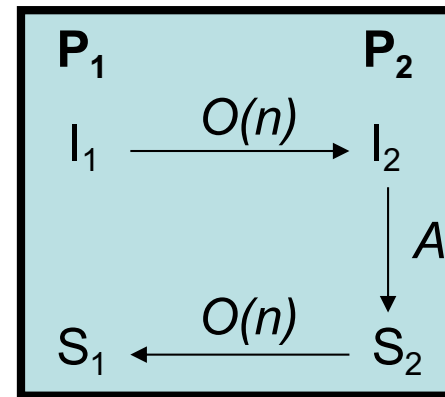


Exemplos

1) Sejam P_1 e P_2 dois problemas tais que $P_1 \propto P_2$ e suponha que P_1 tem cota inferior $\Omega(n \log n)$. Quais das seguintes afirmações são verdadeiras?

- a) $\Omega(n \log n)$ é também uma cota inferior para P_2 ; (V)
- b) Todo algoritmo que resolve P_1 pode ser usado para resolver P_2 ; (F)
- c) Todo algoritmo que resolve P_2 pode ser usado para resolver P_1 ; (V)
- d) O problema P_2 tem complexidade $\theta(n \log n)$. (F)

2) Sejam P_1 e P_2 dois problemas tais que um deles tem cota inferior $\Omega(n^k)$, para algum $k > 1$, e o outro é solucionável em tempo $O(n \log n)$. Se P_1 é redutível a P_2 em tempo linear, quem é quem?



Alternativa 1:

P_1 : $\Omega(n^k)$
 P_2 : $O(n \log n)$

Alternativa 2:

P_1 : $O(n \log n)$
 P_2 : $\Omega(n^k)$

Exemplos

1) Sejam P_1 e P_2 dois problemas tais que $P_1 \propto P_2$ e suponha que P_1 tem cota inferior $\Omega(n \log n)$. Quais das seguintes afirmações são verdadeiras?

- a) $\Omega(n \log n)$ é também uma cota inferior para P_2 ; (V)
- b) Todo algoritmo que resolve P_1 pode ser usado para resolver P_2 ; (F)
- c) Todo algoritmo que resolve P_2 pode ser usado para resolver P_1 ; (V)
- d) O problema P_2 tem complexidade $\theta(n \log n)$. (F)

2) Sejam P_1 e P_2 dois problemas tais que um deles tem cota inferior $\Omega(n^k)$, para algum $k > 1$, e o outro é solucionável em tempo $O(n \log n)$. Se P_1 é redutível a P_2 em tempo linear, quem é quem?

Podemos transferir para P_1 quaisquer limites superiores que sejam conhecidos para P_2

P_1 : $O(n \log n)$

P_2 : $\Omega(n^k)$

$$A \propto B$$

conclusão:

$$I_A \rightarrow I_B$$

$$O(+n)$$

$$S_A \leftarrow S_B$$

$$\Omega(A) \leq \Omega(B) \quad (1)$$

Exemplo: $A \propto B$

$$A \rightarrow O(n \log n) \quad (2)$$

$$B \rightarrow \Omega(n^2) \quad (3)$$

De (2):

$$\Omega(A) \leq \Omega(n \log n) \quad (4)$$

De (1):

$$\Omega(A) \leq \Omega(B); \text{ mas, } \Omega(B) = \Omega(n^2)$$

logo:

$$\Omega(A) \leq \Omega(n^2) \quad (5)$$

Então:

$$\text{de (4)} \rightarrow \Omega(A) \leq \Omega(n \log n)$$

$$\text{de (5)} \rightarrow \Omega(A) \leq \Omega(n^2)$$

$$\text{como } \Omega(n \log n) < \Omega(n^2),$$

$$\underline{\underline{\Omega(A) \leq \Omega(n \log n) < \Omega(n^2) = \Omega(B)}}$$

OK

Considerações

- Na sequência, vamos mostrar que o problema de ordenação pode ser reduzido a diversos problemas importantes de Geometria Computacional. Assim, vamos ser capazes de obter uma cota inferior $\Omega(n \log n)$ para tais problemas, utilizando os resultados dos Teoremas mostrados. Isto nos permitirá discutir a existência de algoritmos ótimos para tais problemas;
- No entanto, nem sempre, é possível utilizar o problema de ordenação para estabelecer cotas inferiores para problemas de interesse em Geometria Computacional. Uma análise mais sofisticada pode ser necessária. Métodos de Geometria Algébrica é muitas vezes empregado para estabelecer cotas inferiores para a complexidade destes problemas.

